

TruthSeeker

AI-Powered Fact-Checking System

Corrective Retrieval-Augmented Generation (CRAG)

Complete Code Walkthrough & Presentation Guide

LangGraph | Groq Llama 3.3 | Supabase pgvector | Tavily | Streamlit

1. What Is This Project?

TruthSeeker is an AI-powered fact-checking system. You give it any claim, and it determines -- using real evidence -- whether that claim is **TRUE**, **FALSE**, or **UNVERIFIABLE**.

It uses **CRAG (Corrective Retrieval-Augmented Generation)**, a smarter version of standard RAG. Instead of blindly trusting whatever documents it finds, it first *grades* those documents for relevance. If they are not good enough, it falls back to the live web for better evidence.

RAG vs CRAG -- Key Difference

Feature	Description
Regular RAG	Retrieve docs then generate -- no quality check
CRAG (This Project)	Retrieve -> Grade relevance -> Decide -> Generate
Fallback	If DB evidence is weak, search live web via Tavily
Source Tracking	Knows if answer came from DB, web, or both

Project File Map

```
AiProject/
|
+-- supabase_setup.sql  <- Step 0: One-time database setup
+-- ingest.py           <- Step 1: Load data -> embed -> store in DB
+-- graph.py           <- Step 2: The AI brain (CRAG pipeline)
+-- app.py              <- Step 3: The web UI (Streamlit frontend)
|
+-- test_graph.py       <- Quick test runner
+-- requirements.txt    <- Python dependencies
+-- .env                <- Secret API keys (not pushed to GitHub)
+-- data/
    +-- True.csv         <- 21,000 real news articles
```

Overall System Flow

```
True.csv (21K articles)
-> [ingest.py] Generate embeddings -> Store in Supabase

User types a claim
-> [app.py] Send to CRAG Graph
-> [Node 1] RETRIEVE: cosine similarity search in Supabase
-> [Node 2] GRADE: LLM evaluates document relevance
If >= 2 docs relevant -> [Node 4] GENERATE verdict
If < 2 docs relevant -> [Node 3] WEB SEARCH via Tavily -> [Node 4] GENERATE
-> [app.py] Display: TRUE / FALSE / UNVERIFIABLE
```

2. supabase_setup.sql -- Database Setup

Run once in the Supabase SQL Editor. Creates the vector database schema and search function.

> Enable pgvector extension

```
CREATE EXTENSION IF NOT EXISTS vector;
```

-> Enables pgvector: a PostgreSQL plugin that stores and searches vector embeddings (lists of numbers representing text meaning).

> Create the documents table

```
CREATE TABLE IF NOT EXISTS documents (  
  id          BIGSERIAL PRIMARY KEY,  
  content     TEXT NOT NULL,  
  metadata    JSONB DEFAULT '{}',  
  embedding   VECTOR(384)  
);
```

Four columns:

- * id -- auto-incremented unique identifier
- * content -- the actual text of a news article chunk
- * metadata -- extra info: title, date, subject (stored as JSON)
- * embedding -- a 384-dimensional vector (mathematical fingerprint of the text)

> Create vector similarity index

```
CREATE INDEX IF NOT EXISTS documents_embedding_idx  
ON documents USING ivfflat (embedding vector_cosine_ops)  
WITH (lists = 100);
```

-> IVFFlat index = approximate nearest-neighbor index. Makes similarity searches fast even with millions of rows.

> Create the match_documents() search function

```
CREATE OR REPLACE FUNCTION match_documents(  
  query_embedding VECTOR(384),  
  match_count INT DEFAULT 5  
)  
RETURNS TABLE (id, content, metadata, similarity FLOAT)  
AS $$  
  SELECT d.id, d.content, d.metadata,  
         1 - (d.embedding <=> query_embedding) AS similarity  
  FROM documents d  
  ORDER BY d.embedding <=> query_embedding  
  LIMIT match_count;  
$$;
```

-> Computes cosine distance (<=> operator) between query vector and all stored document vectors.

-> similarity = 1 - distance (1.0 = perfect match, 0 = completely unrelated).

3. ingest.py -- Data Ingestion Pipeline

A one-time script that reads True.csv (21,000 real news articles), converts them to vectors, and loads everything into Supabase. Must be run before the app is used.

> Configuration constants

```
DATA_PATH      = 'data/True.csv'
CHUNK_SIZE     = 500    # characters per chunk
CHUNK_OVERLAP  = 50     # overlap between consecutive chunks
EMBEDDING_MODEL = 'all-MiniLM-L6-v2'
BATCH_SIZE     = 100    # rows per Supabase insert
```

-> All tunable parameters in one place. CHUNK_SIZE=500 splits articles into focused, digestible pieces.

> Step 1 -- load_dataset()

```
df = pd.read_csv('data/True.csv')
```

-> Loads the Kaggle Fake-and-Real News Dataset. Columns: title, text, subject, date. Contains 21,000 verified real news articles.

> Step 2 -- prepare_documents()

```
full_text = f"{row['title']}\n\n{row['text']}"
metadata = {'subject': ..., 'date': ..., 'title': ...}
```

-> Combines title + body text for richer context. Metadata stored separately for display to users later.

> Step 3 -- chunk_documents()

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50,
    separators=['\n\n', '\n', '. ', ' ', ''],
)
```

-> Splits on paragraph breaks first, then newlines, then periods -- preserving sentence integrity.

-> 50-char overlap ensures context is not lost at chunk boundaries.

> Step 4 -- generate_embeddings()

```
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(texts, show_progress_bar=True, batch_size=64)
```

-> Converts each text chunk into a 384-dimensional vector.

-> KEY: Same model is used at query time so all vectors live in the same mathematical space.

> Step 5 -- insert_into_supabase()

```
for i in range(0, total, batch_size=100):
    batch = [{'content':..., 'metadata':..., 'embedding':...}]
    supabase.table('documents').insert(batch).execute()
    # On failure: retries one record at a time
```

-> Batch inserts of 100 rows to avoid timeouts. Falls back to single-record inserts if a batch fails.

4. graph.py -- The CRAG AI Brain (Most Important File)

Defines the 4-node LangGraph state machine. Every fact-check request runs through this pipeline. This is the core intelligence of TruthSeeker.

A. Imports & Client Setup

```
from langchain_groq import ChatGroq
from sentence_transformers import SentenceTransformer
from supabase import create_client, Client
from tavily import TavilyClient
from langgraph.graph import StateGraph, END

llm = ChatGroq(model='llama-3.3-70b-versatile', temperature=0)
```

Key libraries:

- * LangGraph -- builds and executes the state machine workflow graph
- * ChatGroq -- ultra-fast access to Llama 3.3 70B via Groq Cloud API
- * SentenceTransformer -- generates 384-dim vectors at query time
- * Supabase client -- queries the vector DB with RPC calls
- * Tavily -- live web search restricted to trusted news domains

-> temperature=0: deterministic output -- no randomness. Same question always gives same answer. Critical for a fact-checker.

B. Portable Secret Reader

```
def _get_secret(key: str) -> str:
    try:
        import streamlit as st
        if key in st.secrets: return st.secrets[key]
    except Exception: pass
    return os.getenv(key) # falls back to .env file
```

-> Works both locally (reads .env) AND on Streamlit Cloud (reads st.secrets). Zero code change needed.

C. Rate Limit Handler

```
def invoke_with_retry(chain, inputs, max_retries=3):
    for attempt in range(max_retries):
        try:
            return chain.invoke(inputs)
        except Exception as e:
            if '429' in str(e): # rate limited
                time.sleep(15 * (attempt + 1)) # 15s, 30s, 45s
```

-> If Groq API returns HTTP 429 (rate limit exceeded), waits 15 seconds then retries. Up to 3 attempts.

D. GraphState -- Shared Memory

```
class GraphState(TypedDict):
    claim: str # Input: the user's claim text
    documents: List[Document] # Retrieved / web-searched docs
    web_search_needed: str # 'yes' | 'no'
    source_type: str # 'database' | 'web' | 'both'
    generation: str # Final verdict text
    route_log: List[str] # Audit trail of every step taken
```

- > The shared memory of the graph. Every node reads from it and writes back to it.
- > route_log acts like a flight recorder -- tracks every step. Shown to users in the UI.

4. graph.py -- The 4 Nodes

NODE 1: retrieve() -- Database Similarity Search

```
claim_embedding = embedding_model.encode(claim).tolist()

response = supabase.rpc('match_documents', {
    'query_embedding': claim_embedding,
    'match_count': 5,
}).execute()
```

- * Convert claim string -> 384-dim vector (same model as ingestion)
- * Call match_documents() SQL function via Supabase RPC
- * Get top-5 documents with highest cosine similarity scores
- * Wrap results as LangChain Document objects with similarity metadata

NODE 2: grade_documents() -- LLM Relevance Check

```
GRADING_PROMPT system: 'You are a relevance grader...'
'Respond with comma-separated IDs of relevant docs'
'If NONE are relevant, reply exactly: NONE'

# LLM Response examples: '0, 2, 4' or 'NONE'

web_search_needed = 'yes' if len(relevant_docs) < 2 else 'no'
```

- > LLM reads all 5 docs in one batch call and picks which ones are actually relevant to the claim.
- > KEY CRAG DECISION: if fewer than 2 relevant docs found, trigger web search fallback.

NODE 3: web_search() -- Live Web Fallback

```
search_results = tavily_client.search(
    query=f'fact check: {claim}',
    max_results=5,
    include_domains=[
        'reuters.com', 'apnews.com', 'bbc.com',
        'snopes.com', 'factcheck.org', 'politifact.com'
    ]
)
```

- > Only activated when database alone does not have enough relevant evidence.
- > Restricted to trusted fact-checking and news domains only -- no random blogs or social media.
- > Combines web results with any existing DB docs (source_type = 'both').

NODE 4: generate() -- Verdict Production

```
System prompt: 'You are TruthSeeker, an expert fact-checking AI...'
'CURRENT SYSTEM DATE is 100% accurate and authoritative.'
'FORMAT YOUR RESPONSE EXACTLY AS:'
'  VERDICT:    [TRUE/FALSE/UNVERIFIABLE]'
'  STANCE:     [SUPPORT/REFUTE/NEUTRAL]'
'  REASONING:  [2-4 sentence explanation]'

Human: '[CURRENT DATE: {current_date}]'
      'CLAIM: {claim}'
      'EVIDENCE DOCUMENTS: {evidence}'
```

- > LLM is given the current system date to verify date-based claims (e.g. 'today is Monday').
- > Structured output format is enforced via the system prompt -- easy to parse in app.py.

Conditional Edge: `decide_route()`

```
def decide_route(state: GraphState) -> str:
    if state.get('web_search_needed') == 'yes':
        return 'web_search'
    return 'generate'
```

- > The router -- the 'Corrective' part of CRAG. Called by LangGraph after grading to decide next node.

4. graph.py -- Assembling the Pipeline

build_graph() -- Wires Everything Together

```
workflow = StateGraph(GraphState)

# Register all 4 nodes
workflow.add_node('retrieve', retrieve)
workflow.add_node('grade_documents', grade_documents)
workflow.add_node('web_search', web_search)
workflow.add_node('generate', generate)

# Entry point
workflow.set_entry_point('retrieve')

# Fixed edges
workflow.add_edge('retrieve', 'grade_documents')
workflow.add_edge('web_search', 'generate')
workflow.add_edge('generate', END)

# Conditional edge (the CRAG decision point)
workflow.add_conditional_edges(
    'grade_documents', decide_route,
    {'web_search': 'web_search', 'generate': 'generate'}
)

app = workflow.compile()
return app
```

-> The entire pipeline declared in ~15 lines. LangGraph handles all execution, state passing, and routing.

5. app.py -- Streamlit Frontend

The web interface users interact with. Calls graph.py and displays results beautifully.

A. Page Config & Custom CSS (~200 lines)

- * background: dark space gradient (#0a0a1a) -- premium dark mode feel
- * .hero-title -- large gradient text (cyan -> purple -> red)
- * .verdict-true / false / unverifiable -- color-coded result cards (green/red/yellow)
- * .route-step -- audit log entries with purple left border
- * @keyframes fadeInUp -- smooth card entrance animation

B. parse_verdict() -- LLM Output Parser

```
for line in lines:
    if line.startswith('VERDICT:'):
        result['verdict'] = ... # 'TRUE'/'FALSE'/'UNVERIFIABLE'
    elif line.startswith('STANCE:'):
        result['stance'] = ... # 'SUPPORT'/'REFUTE'/'NEUTRAL'
    elif line.startswith('REASONING:'):
        result['reasoning'] = ...
```

-> Parses the LLM's structured text output into a Python dict for rendering in the UI.

C. Cached Graph Loading

```
@st.cache_resource
def load_graph():
    return build_graph()
```

-> Graph is built ONCE per app session -- not on every button click. Saves ~2-3 seconds per query.

D. Main UI Flow

```
# 1. Hero header  2. Claim text input  3. Submit button

if submit and claim.strip():
    result = graph.invoke({'claim': claim, 'documents': [], ...})
    parsed = parse_verdict(result['generation'])

    # 4. Color-coded verdict card (green / red / yellow)
    # 5. Reasoning explanation box
    # 6. Expandable: source documents used as evidence
    # 7. Expandable: agent routing audit log
```

6. requirements.txt -- Python Dependencies

Library	Role
langchain, langchain-core	LLM prompt templates, document objects, parsers
langchain-groq	Connects LangChain to Groq's Llama 3.3 API
langgraph	State machine / graph execution engine
streamlit	Web UI framework
supabase	Supabase Python client (DB + RPC calls)
sentence-transformers	all-MiniLM-L6-v2 embedding model
pandas	Loading and processing the CSV dataset
python-dotenv	Loading .env API key file
tavily-python	Live web search API client

7. test_graph.py -- Quick Smoke Test

```
result = graph.invoke({
    'claim': 'today is monday',
    'documents': [], 'web_search_needed': 'no',
    'source_type': 'database', 'generation': '', 'route_log': [],
})
print(result['generation'])
print(result['route_log'])
```

-> 23-line script to verify the pipeline works end-to-end. Tests a simple date claim whose answer the LLM can verify using the system date.

8. Key Concepts Summary

The Embedding Pipeline

```
'NASA found water on Mars'
  -> all-MiniLM-L6-v2 model
[0.02, -0.14, 0.87, ... 384 numbers] (the vector)
  -> cosine similarity search against all stored vectors
'NASA announces water ice on Mars' -> similarity: 0.91 (MATCH)
'Apple releases new iPhone model'  -> similarity: 0.12 (no match)
```

Three Possible Verdicts

TRUE -- Evidence (or system date) confirms the claim

FALSE -- Evidence (or system date) contradicts the claim

UNVERIFIABLE -- Evidence is insufficient to decide either way

How to Run the Project

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Create .env with your 4 API keys
#   SUPABASE_URL, SUPABASE_KEY, GROQ_API_KEY, TAVILY_API_KEY

# 3. Run SQL setup in Supabase dashboard (one-time)
#   supabase_setup.sql

# 4. Ingest data into Supabase (one-time, ~10-20 minutes)
python ingest.py

# 5. Launch the web app
streamlit run app.py
```